

REVIVE - Seções revisadas para a Fase 02

ENTREGA 2 - Perguntas objetivas

Não há novas perguntas indispensáveis. As informações pendentes foram confirmadas: a versão da nova entrada será 1.2, a data será 19/04/2026, o schema Supabase informado será usado como referência, o arquivo compactado local será ignorado e as inconsistências encontradas na navegação serão mencionadas.

ENTREGA 3 - Texto final revisado

PARTE 00 - HISTÓRICO DE VERSÃO DO PROJETO

| Versão | Período | Fase | Data | Descrição |

|---|---:|---:|---|---|

| 1.0 | 1º | 01 | 08/03/2026 | Criação inicial do documento do projeto, definição do nome do produto, descrição geral, identificação da equipe e planejamento do semestre. |

| 1.1 | 1º | 02 | 19/04/2026 | Ajustes na descrição do produto, inclusão dos objetivos do semestre, revisão do planejamento e primeiros refinamentos a partir do feedback recebido. |

| 1.2 | 1º | 02 | 19/04/2026 | Revisão das seções de planejamento, arquitetura, implementação por camadas, qualidade de software e segurança da informação. Nesta versão, o documento foi comparado com o código real do projeto, os testes automatizados foram executados localmente e foram registrados pontos de melhoria identificados na navegação, na cobertura de testes e nos controles de segurança. |

PARTE 02 - PLANEJAMENTO DO SEMESTRE

5 SITUAÇÃO ATUAL DO PROJETO

O projeto REVIVE encontra-se em uma fase funcional de desenvolvimento, com backend, frontend e banco de dados já integrados. A aplicação atual é uma plataforma web para acompanhamento da jornada de recuperação de vícios, permitindo cadastro e login de usuário, registro de vícios, acompanhamento de dias de abstinência, cálculo de economia financeira, registro diário de humor e observações, histórico de recaídas, metas pessoais, conquistas, relatórios e visualização de informações em dashboard.

O backend foi implementado em Node.js com Express.js e concentra as rotas da API REST no arquivo `index.js`. A API utiliza autenticação com JWT, armazenamento de senhas com bcrypt, limitação de requisições com `express-rate-limit`, configuração de CORS, logs com Morgan e documentação Swagger/OpenAPI em `/api/docs`. As rotas principais cobrem autenticação, perfil do usuário, vícios, registros diários, recaídas, metas e mensagens motivacionais. O acesso ao banco é feito por meio da biblioteca `@supabase/supabase-js`.

O banco de dados está estruturado no Supabase/PostgreSQL. O schema informado contém as tabelas `usuarios`, `vicios`, `registros_diarios`, `historico_recaidas`, `metas`, `mensagens_motivacionais` e `marcos`. No código atual, as seis primeiras tabelas são utilizadas diretamente pela aplicação. A tabela `marcos` existe no schema, mas não foi encontrada integração direta dela nas rotas ou telas atuais, portanto deve ser tratada como possibilidade de evolução e não como funcionalidade já implementada.

O frontend foi desenvolvido com React, Vite e Tailwind CSS. A aplicação possui páginas de Login, Dashboard, Analytics, Detalhes, Metas, Calendário, Conquistas, Relatórios, Perfil e Dicas. A organização segue uma separação por páginas, componentes, contextos, serviços, utilitários e configuração de ambiente. O estado principal fica distribuído entre `AuthContext`, `DataContext` e `UIContext`, enquanto a comunicação com a API é encapsulada nos arquivos da pasta `services`.

Durante a revisão da Fase 02, foram identificados pontos que precisam de ajuste. O botão “Novo” da barra de navegação aponta para `/novo-vicio`, mas essa rota não existe no roteamento atual; na prática, o cadastro de novo vício acontece pelo wizard exibido no Dashboard. Também foi encontrada uma diferença de nomenclatura na barra de navegação: o componente tenta ler `vicioSelecionado`, enquanto o contexto de dados expõe `selectedAddiction`. Isso não compromete o funcionamento central do

sistema, mas impede que o item dinâmico de detalhes funcione como descrito nos comentários do código.

O projeto já possui testes automatizados com Vitest, Supertest e React Testing Library. A validação local executada com ``npm run validate`` passou, incluindo testes da API, testes do painel e build de produção do frontend. Apesar disso, a cobertura ainda é limitada e se concentra em funções auxiliares, validações iniciais e alguns comportamentos de interface. Fluxos críticos, como cadastro completo, login real com banco, criação de vício, recaída, metas e autorização entre usuários, ainda precisam de testes mais abrangentes.

6 OBJETIVOS DO SEMESTRE

Para esta fase, o foco do projeto deixa de ser apenas implementar telas e rotas e passa a incluir estabilização, validação e análise crítica do que já foi construído. Os objetivos do semestre são:

1. consolidar as funcionalidades principais já implementadas, principalmente autenticação, gerenciamento de vícios, registros diários, recaídas, metas e relatórios;
2. corrigir inconsistências encontradas durante a revisão, como a rota inexistente ``/novo-vicio`` e a diferença de nomenclatura entre a navegação e o contexto de dados;
3. ampliar os testes automatizados para cobrir fluxos completos da API e da interface, principalmente cadastro, login, criação de vício, registro diário, recaída e metas;
4. registrar testes manuais de navegação e uso do sistema, pois atualmente o repositório tem evidência automatizada, mas não possui roteiro formal de testes manuais;
5. revisar a segurança da informação, com atenção ao uso de JWT em ``localStorage``, validação de entrada, isolamento dos dados por usuário e configuração real do Supabase;
6. documentar a arquitetura real do sistema com base no código, evitando tratar comentários antigos ou afirmações do documento como verdade sem validação;
7. preparar o projeto para uma entrega mais madura, com ``env.example``, documentação de execução, evidências de teste e, se possível, scripts ou migrations do banco.

Esses objetivos foram definidos porque o sistema já possui uma base funcional, mas ainda precisa de ajustes de qualidade para reduzir risco de falhas em fluxos importantes. A revisão mostrou que a aplicação não está apenas em fase de protótipo visual, porém também não deve ser descrita como finalizada ou totalmente validada.

7 CRONOGRAMA DO SEMESTRE

O cronograma do semestre pode ser organizado em seis etapas:

****Análise e diagnóstico (Semanas 1-2):**** revisão do documento, conferência do código real, identificação das funcionalidades implementadas, verificação da estrutura do banco e levantamento de inconsistências entre documentação e projeto.

****Correções funcionais (Semanas 3-5):**** ajuste da navegação para criação de novo vício, correção da nomenclatura usada entre `NavBar` e `DataContext`, revisão de mensagens de erro e melhoria de pequenos comportamentos da interface.

****Testes e validação (Semanas 6-8):**** ampliação dos testes automatizados da API e do frontend, criação de roteiros de testes manuais e execução dos fluxos principais: cadastro, login, criação de vício, registro diário, recaída, metas, relatórios e exportação.

****Segurança e banco de dados (Semanas 9-10):**** revisão das permissões no Supabase, documentação das políticas de acesso, avaliação do armazenamento do token, validação dos dados recebidos pela API e criação de arquivo `.env.example` sem chaves reais.

****Documentação e preparação de entrega (Semanas 11-13):**** atualização do documento de especificação, revisão da documentação Swagger, melhoria do README, organização das evidências de teste e preparação do build do frontend.

****Validação final (Semanas 14-16):**** execução completa da validação local, testes finais de uso, correção de problemas encontrados na entrega, revisão da apresentação e fechamento da documentação acadêmica.

20 ARQUITETURA GERAL DO SISTEMA

O REVIVE utiliza uma arquitetura cliente-servidor em três camadas: apresentação, aplicação e dados. Essa divisão foi confirmada no código e ajuda a separar a interface do usuário, as regras da API e o armazenamento das informações.

A camada de apresentação é uma SPA desenvolvida em React com Vite. Ela é responsável pela navegação, telas, formulários, feedback visual, gráficos, relatórios e organização da experiência do usuário. O frontend não acessa diretamente o Supabase. Toda comunicação externa passa pela camada de serviços em ``src/services``, que utiliza ``fetch`` para chamar a API REST. O token JWT é armazenado no ``localStorage`` com a chave ``revive_token`` e enviado no cabeçalho ``Authorization: Bearer <token>`` nas requisições protegidas.

A camada de aplicação é uma API REST construída com Express.js. Ela centraliza autenticação, autorização, validações iniciais, cálculo de estatísticas e comunicação com o banco. O backend implementa rotas para cadastro, login, perfil, vícios, registros diários, recaídas, metas e mensagens motivacionais. Também possui middleware de autenticação JWT, CORS, rate limiting, logs HTTP com Morgan e documentação Swagger/OpenAPI.

A camada de dados utiliza Supabase com PostgreSQL. O schema informado contém tabelas para usuários, vícios, registros diários, histórico de recaídas, metas, mensagens motivacionais e marcos. A aplicação atual utiliza diretamente as tabelas ``usuarios``, ``vicios``, ``registros_diarios``, ``historico_recaidas``, ``metas`` e ``mensagens_motivacionais``. A tabela ``marcos`` não aparece integrada no código analisado, então não deve ser apresentada como funcionalidade ativa.

O fluxo principal começa quando o usuário acessa o frontend e realiza login ou cadastro. No login, o backend consulta a tabela ``usuarios``, compara a senha recebida com o hash bcrypt armazenado e retorna um JWT quando as credenciais são válidas. Depois disso, o frontend usa esse token para carregar vícios, metas, registros, recaídas

e mensagens. Ao listar vícios, o backend calcula dias de abstinência, tempo formatado e valor economizado antes de devolver os dados ao frontend.

Depois da revisão de testes e segurança, a arquitetura continua adequada para o porte atual do projeto, mas alguns cuidados precisam ser registrados. A API faz o isolamento dos dados por ``usuario_id`` em várias rotas, o que reduz risco de acesso indevido. Porém, o repositório não contém migrations nem políticas RLS do Supabase, então não é possível afirmar pelo código local que o banco está protegido independentemente da API. Também foi observado que a sanitização atual é básica, baseada principalmente em ``trim()``, e não deve ser descrita como proteção completa contra XSS ou payloads maliciosos.

22 IMPLEMENTAÇÃO POR CAMADAS DO SISTEMA

22.1 Frontend

O frontend do REVIVE foi implementado como uma Single Page Application em React, com build e servidor de desenvolvimento pelo Vite. A estrutura está dividida em páginas, componentes, contextos, serviços, configuração e utilitários. As páginas principais são: Login, Dashboard, Analytics, Detalhes, Metas, Calendário, Conquistas, Relatórios, Perfil e Dicas.

O roteamento é feito com React Router. As rotas autenticadas ficam dentro do componente ``ProtectedRoute``, que verifica se existe token e usuário no contexto de autenticação antes de liberar o acesso. O layout autenticado usa ``AppShell``, com ``Header``, ``NavBar``, ``ToastContainer``, ``ConfirmModal``, ``Alert`` e ``ErrorBoundary``. Essa composição permite reaproveitar a mesma estrutura visual em todas as telas internas.

O estado da aplicação é organizado em três contextos. O ``AuthContext`` controla login, cadastro, logout e verificação inicial do token. O ``DataContext`` carrega e altera vícios, registros, metas, recaídas e mensagem motivacional. O ``UIContext`` concentra tema, toasts, alertas, loading e modal de confirmação.

A comunicação com a API foi separada nos serviços ``auth.service.js``, ``vicios.service.js``, ``registros.service.js``, ``metas.service.js``, ``recaidas.service.js``, ``mensagens.service.js`` e ``api.js``. Essa separação facilita a manutenção, porque as telas não montam

manualmente todas as requisições. A constante ``API_BASE`` é lida de ``VITE_API_URL``, com fallback para ``http://localhost:3000/api``.

O frontend também possui recursos complementares importantes, como exportação CSV na página de relatórios, exportação JSON no perfil, tema claro/escuro, gráficos e indicadores visuais, além de um Service Worker básico. Esse Service Worker faz cache do shell da aplicação e de assets locais, mas não implementa sincronização offline de dados. Por isso, ele deve ser descrito como suporte offline parcial.

Na revisão da implementação, foram encontrados dois ajustes necessários. O primeiro é o botão “Novo” da barra de navegação, que envia o usuário para ``/novo-vicio``, uma rota que não está registrada no ``App.jsx``. O segundo é a tentativa da ``NavBar`` de usar ``vicioSelecionado``, enquanto o contexto trabalha com ``selectedAddiction``. O cadastro de novo vício funciona pelo wizard do Dashboard, mas a navegação precisa ser alinhada para evitar caminho quebrado.

22.2 Backend

O backend foi implementado em Node.js com Express.js, concentrando a API REST no arquivo ``index.js``. A aplicação carrega variáveis de ambiente com ``dotenv``, configura CORS, interpreta JSON, registra logs com Morgan, aplica rate limiting e expõe a documentação Swagger em ``/api/docs``.

As rotas de autenticação permitem cadastro e login. No cadastro, o sistema valida campos obrigatórios, exige senha com no mínimo seis caracteres, pelo menos uma letra maiúscula e pelo menos um caractere especial, verifica se o e-mail já existe e salva a senha com hash bcrypt. No login, a API busca o usuário pelo e-mail, compara a senha com bcrypt e gera um JWT com validade de sete dias.

As rotas protegidas usam um middleware que extrai o token do cabeçalho ``Authorization``, valida a assinatura com ``JWT_SECRET`` e adiciona o identificador do usuário à requisição. A partir disso, as rotas de negócio filtram os dados pelo usuário autenticado quando necessário.

As funcionalidades de vícios permitem listar, criar, consultar detalhes e excluir. Na listagem e no detalhe, o backend calcula dias de abstinência, valor economizado e duração formatada. A exclusão de vício remove registros dependentes de `registros\_diarios`, `historico\_recaidas` e `metas` antes de apagar o vício. As recaídas podem ser registradas com ou sem reset do contador. Os registros diários armazenam humor, gatilhos, conquistas e observações. As metas podem ser criadas, listadas, concluídas e excluídas.

O backend apresenta uma base funcional consistente, mas existem limitações que precisam ser tratadas. A validação de entrada ainda é simples em vários pontos, e a função `sanitize()` apenas remove espaços nas extremidades. Também há uma rota de criação de meta que aceita `vicio\_id` sem verificar no próprio handler se o vício pertence ao usuário, dependendo mais da consistência do banco e da filtragem posterior. Para uma versão mais segura, essa verificação deveria ser adicionada.

22.3 Integração

A integração entre frontend e backend ocorre por requisições HTTP em JSON. O frontend chama os serviços, os serviços chamam `apiCall`, e `apiCall` adiciona o token quando necessário. O backend recebe as requisições, valida autenticação e dados básicos, consulta o Supabase e retorna respostas também em JSON.

No fluxo de autenticação, o usuário envia e-mail e senha pela tela de login. A API valida as credenciais e retorna o token. O frontend armazena esse token no `localStorage` e passa a usá-lo nas próximas chamadas. Ao recarregar a aplicação, o frontend tenta validar o token chamando a API. Esse comportamento funciona como persistência de sessão, mas ainda pode ser melhorado, pois a restauração atual não busca o perfil completo do usuário e pode exibir um usuário genérico.

Os testes executados confirmaram que a API responde ao health check, valida campos obrigatórios em login e cadastro, executa funções auxiliares e permite que partes do frontend sejam renderizadas com dados simulados. Porém, a integração completa com o Supabase ainda depende de testes mais amplos, principalmente para fluxos com banco real ou banco de teste controlado.

24 QUALIDADE DO SOFTWARE E TESTES

A estratégia de testes do REVIVE foi estruturada em camadas, usando Vitest como ferramenta principal. No backend, existem testes unitários para funções auxiliares e testes de integração para rotas Express com Supertest. No frontend, existem testes unitários de utilitários e testes de integração com React Testing Library para contextos e páginas.

Na revisão da Fase 02, foi executado o comando ``npm run validate``. Esse comando roda os testes da API, os testes do painel e o build de produção do frontend. O resultado foi positivo: todos os testes passaram e o build foi gerado com sucesso.

Os testes unitários da API estão no arquivo ``tests/unit/helpers.test.js``. Eles verificam a função ``sanitize()``, a formatação de duração em dias e o cálculo de estatísticas de abstinência e economia. Esses testes se relacionam principalmente com os requisitos de acompanhamento de progresso e cálculo de economia, pois validam parte da regra que transforma dados do vício em informações exibidas ao usuário.

Os testes de integração da API estão em ``tests/integration/routes.test.js``. Eles testam o endpoint ``GET /api/health`` e a validação de campos obrigatórios em ``POST /api/auth/login`` e ``POST /api/auth/cadastro``. Esses testes se relacionam com o requisito de autenticação, pois verificam se a API rejeita requisições incompletas antes de acessar o banco.

No frontend, o teste ``formatters.test.js`` valida a função de cálculo de tempo decorrido exibida na interface. O teste de ``AuthContext`` verifica se, após o login simulado, o token e os dados do usuário são armazenados no estado e no ``localStorage``. O teste de ``DataContext`` valida o tratamento de erro quando o carregamento de vícios falha. O teste da ``ReportsPage`` verifica se a página de relatórios renderiza dados agregados vindos do contexto.

Como evidência objetiva da execução, a validação local apresentou os seguintes resultados: 3 testes unitários da API aprovados, 3 testes de integração da API aprovados, 5 testes unitários do frontend aprovados, 3 testes de integração do frontend aprovados e build de produção concluído. No total, foram 14 testes automatizados aprovados, além da compilação do frontend.

Apesar do resultado positivo, a cobertura atual ainda é limitada. Os testes não executam um fluxo completo de usuário com banco real, como cadastro, login, criação de vício, registro diário, criação de meta e registro de recaída. Também não há teste automatizado para autorização entre usuários diferentes, nem teste específico para rate limiting, CORS, Service Worker ou proteção contra entradas maliciosas. Isso significa que os testes atuais ajudam a detectar regressões pequenas, mas ainda não comprovam a estabilidade completa do sistema.

Também não foi encontrado no repositório um roteiro formal de testes manuais. Pela natureza do sistema, os fluxos manuais mais importantes seriam: criar conta, fazer login, cadastrar um vício, registrar humor do dia, criar meta, registrar recaída sem resetar, registrar recaída resetando contador, consultar calendário, exportar relatório CSV, exportar dados JSON e testar logout. Esses fluxos devem ser documentados e executados na próxima etapa, com registro de resultado esperado e resultado obtido.

Em relação à qualidade interna, o projeto apresenta pontos positivos: separação entre serviços e telas no frontend, uso de contextos para estado compartilhado, `ErrorBoundary` no layout autenticado, tratamento de loading e toasts, documentação Swagger da API e pipeline de validação local. Por outro lado, a concentração de todo o backend em um único arquivo torna a manutenção mais difícil à medida que o projeto cresce. Uma evolução natural seria separar rotas, controllers, middlewares e serviços em arquivos próprios.

Com base na execução realizada, o estado atual da qualidade é satisfatório para uma entrega intermediária de faculdade, mas ainda não deve ser tratado como cobertura completa. A suíte confirma partes importantes do sistema, porém precisa evoluir para cobrir fluxos reais e regras de segurança antes de uma entrega final ou deploy público.

25 SEGURANÇA DA INFORMAÇÃO

A análise de segurança do REVIVE foi feita com base no código atual, nas dependências usadas e no schema Supabase informado. O sistema já possui algumas medidas importantes, mas também apresenta riscos que precisam ser tratados com prioridade proporcional ao impacto.

No armazenamento de senhas, o backend utiliza bcrypt com 10 salt rounds. Isso é uma medida adequada para não armazenar senhas em texto puro. No login, a comparação é feita com `bcrypt.compare()`, e o retorno de erro para credenciais inválidas é genérico, o que reduz o risco de enumeração de usuários.

A autenticação é feita com JWT. O token expira em sete dias e é validado pelo middleware antes das rotas protegidas. Esse modelo funciona para a aplicação atual, mas o frontend armazena o token no `localStorage`. O risco é médio, porque um ataque XSS poderia permitir roubo do token. Como mitigação, seria melhor avaliar cookies `HttpOnly` e `Secure` em uma versão de produção, além de aplicar uma política de CSP e revisar pontos de entrada de HTML.

A autorização é tratada em várias rotas por meio do filtro `usuario_id`. Por exemplo, a listagem e consulta de vícios filtram pelo usuário autenticado, e os registros/recaídas usam consultas relacionadas ao vício do usuário. Isso reduz risco de IDOR. Porém, a criação de meta aceita `vicio_id` opcional sem validar explicitamente se o vício pertence ao usuário. Esse risco é médio, porque pode gerar vínculo incorreto se o banco não impedir. A mitigação recomendada é validar o `vicio_id` antes de inserir a meta.

O acesso ao Supabase é feito pelo backend, e o frontend não usa diretamente as chaves do banco. Isso é positivo. Porém, o repositório não contém migrations nem políticas RLS. O schema informado mostra tabelas, chaves primárias, chaves estrangeiras e e-mail único, mas não mostra políticas de Row Level Security. Portanto, não é possível afirmar pelo material analisado que o isolamento também está garantido no banco. O risco é médio caso a chave usada tenha permissões amplas. A mitigação recomendada é documentar e versionar as políticas RLS ou garantir que apenas o backend tenha acesso privilegiado.

O CORS está configurado com lista de origens permitidas pela variável `ALLOWED_ORIGINS`, com fallback para localhost em desenvolvimento. Isso reduz exposição indevida da API em navegadores. O rate limiting também está implementado: rotas de autenticação têm limite mais restrito, e as demais rotas `/api` possuem limite geral. Essa proteção reduz risco de força bruta e abuso, mas não substitui monitoramento ou bloqueio mais avançado em produção.

A validação de entrada existe em pontos importantes, como cadastro, login, criação de vício, criação de registro e criação de meta. Mesmo assim, ela ainda é básica. A função

``sanitize()`` apenas converte para string e aplica ``trim()``. Isso não deve ser descrito como proteção completa contra XSS ou injection. O risco é médio, principalmente porque campos como observações, gatilhos, conquistas e motivo de recaída aceitam texto livre. Como mitigação, recomenda-se validar tamanhos máximos, normalizar tipos esperados, escapar conteúdo na saída e usar biblioteca específica de sanitização quando necessário.

Quanto a SQL injection, o risco é baixo a médio. O projeto usa o SDK do Supabase, evitando montagem direta de SQL bruto na maior parte do código. Porém, a rota de mensagem motivacional usa o parâmetro ``tipo_vicio`` em uma expressão ``or(...)``. Ainda que o SDK reduza o risco de SQL injection tradicional, esse campo deveria ser validado contra valores esperados antes de ser usado na query.

O tratamento de erros foi parcialmente pensado. A função ``sendInternalServerError()`` oculta detalhes em produção, mas inclui ``detalhes`` quando ``NODE_ENV`` não é ``production``. Isso é útil durante desenvolvimento, mas deve ser conferido antes de deploy. Se o ambiente de produção for configurado incorretamente, detalhes internos podem aparecer para o usuário.

Não foi identificado risco alto comprovado no código analisado, considerando que o arquivo ``.env`` está ignorado pelo Git e que as chaves não aparecem versionadas. Os principais riscos atuais são médios e estão ligados ao token no ``localStorage``, à sanitização limitada, à ausência de evidência local de RLS e a algumas validações de autorização que ainda podem ser reforçadas.

Como conclusão, o REVIVE possui uma base inicial de segurança coerente para o estágio do projeto: senha com hash, JWT, rotas protegidas, CORS e rate limiting. Mesmo assim, o sistema ainda não deve ser descrito como seguro de forma absoluta. Para uma entrega final mais madura, é necessário revisar permissões do Supabase, melhorar validações de entrada, corrigir a criação de metas com ``vicio_id``, criar ``.env.example`` com placeholders e avaliar uma estratégia mais segura para armazenamento do token no frontend.